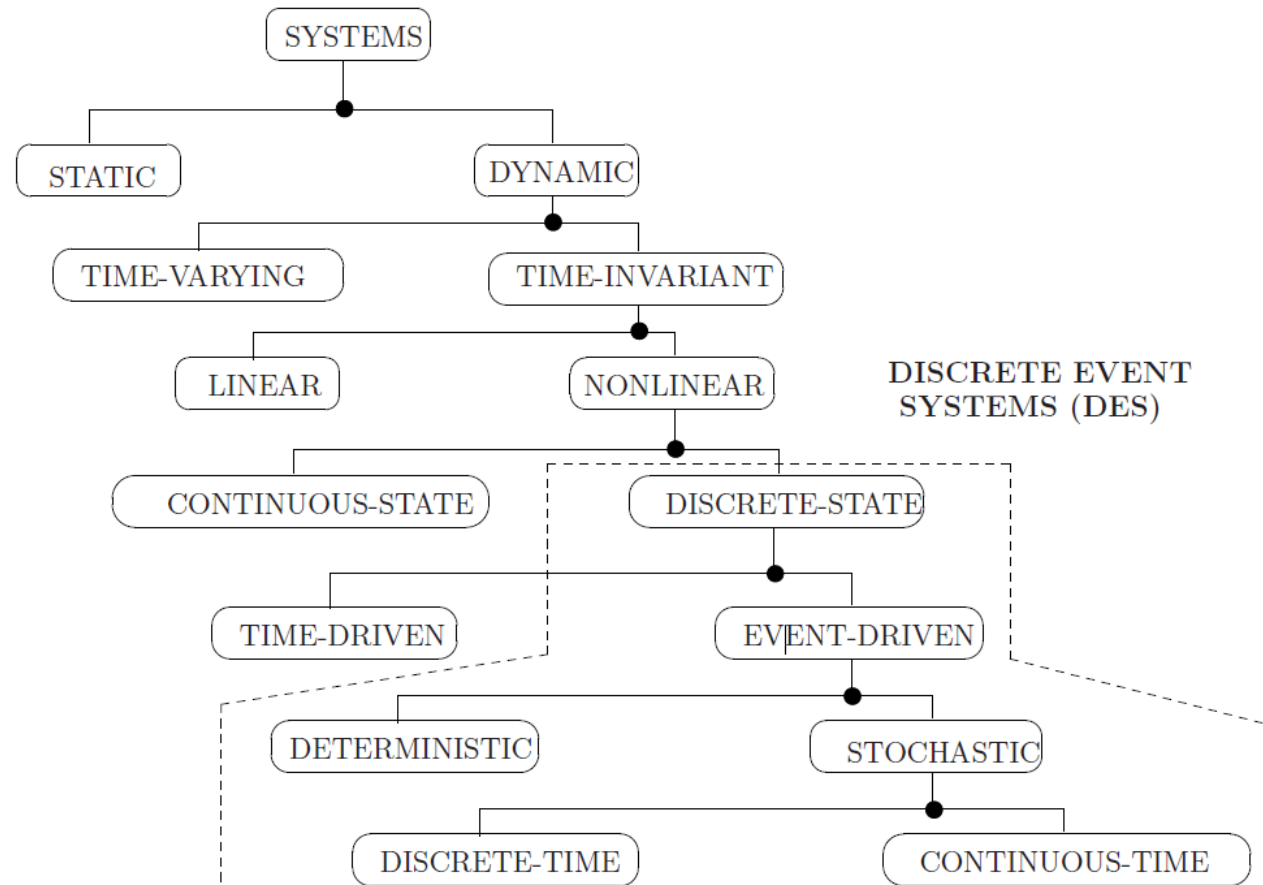


# Discrete Event Systems (DES) Simulation

Henrik Schiøler

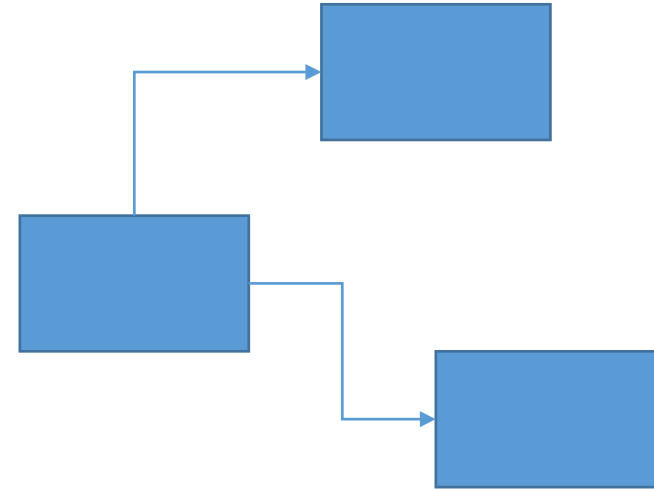
# Systems model taxonomy



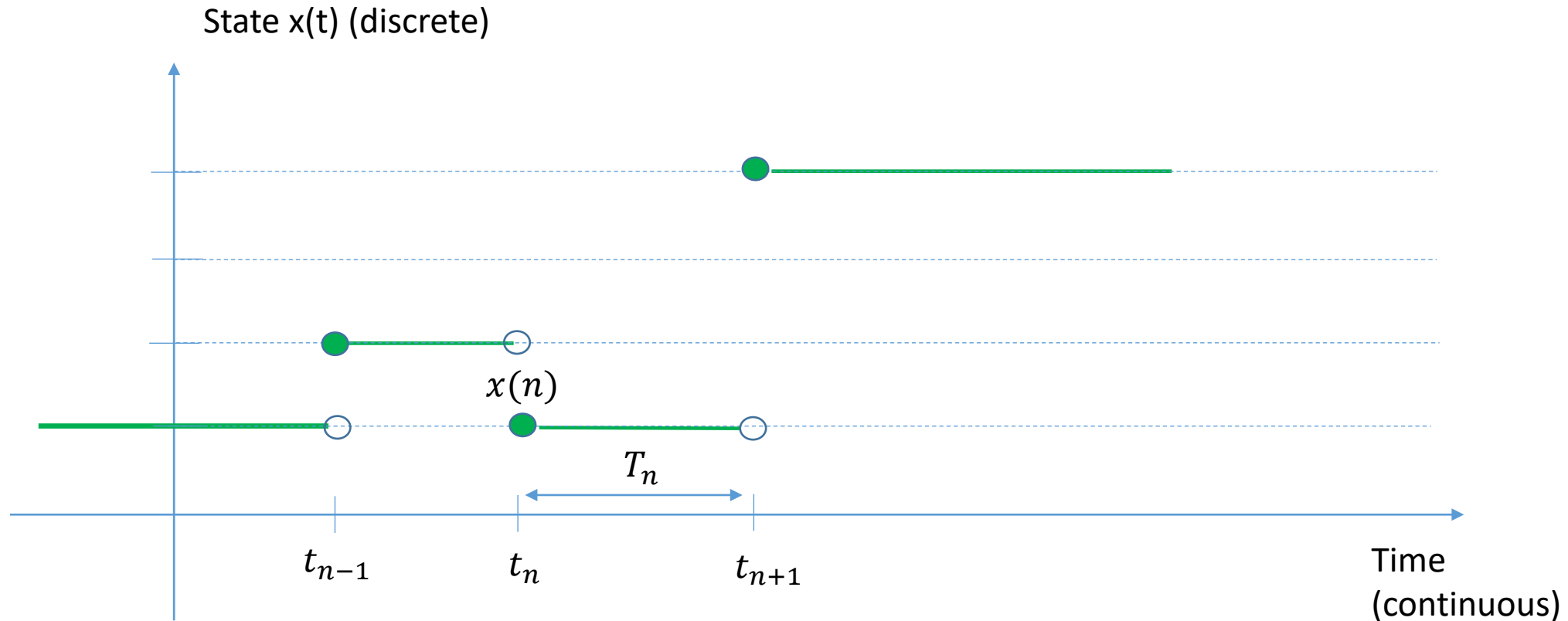
**Figure 1.31:** Major system classifications.

# Timed Automaton

- Finite State Machine
- Transitions triggered by Events
- Events scheduled by timers at transition instants
- At transition instant a future instant is selected for an event to happen
- Selecting future instants may be deterministic or random
- Future events/instants are stored in an ordered list, where earlier events are above later events (**event list**)



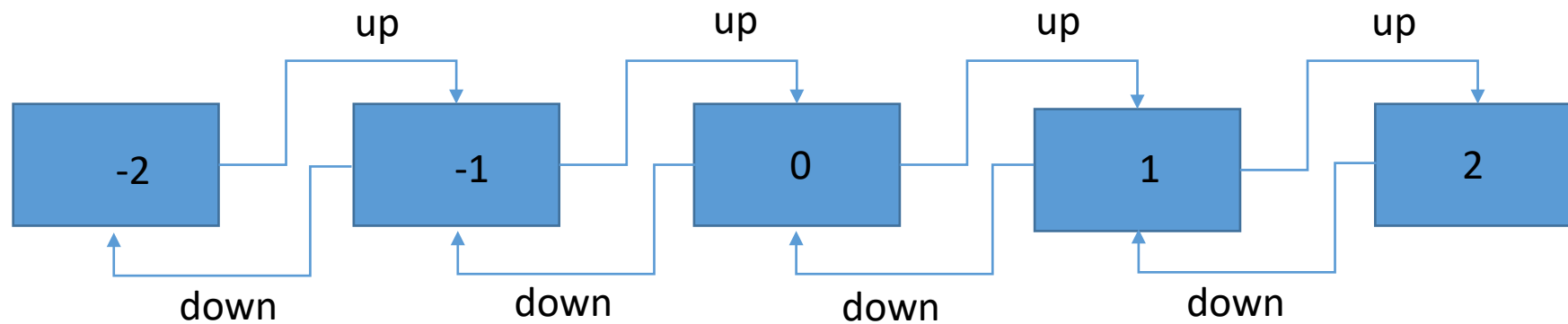
# Timing diagram



Continuity at least from on side

Cadlag: “continues à droite, limites à gauche”; “rcll”

# Example



- State space  $\{-2, -1, 0, 1, 2\}$
- Event set  $\{\text{up}, \text{down}\}$

top



| event | time  |
|-------|-------|
| up    | 11.50 |
| up    | 11.55 |
| down  | 12.10 |

# Stochastic Timed Automaton

**Definition.** A *Stochastic Timed Automaton* is a six-tuple

$$(\mathcal{E}, \mathcal{X}, \Gamma, p, p_0, G)$$

where

$\mathcal{E}$  is a countable *event set*

$\mathcal{X}$  is a countable *state space*

$\Gamma(x)$  is a set of *feasible* or *enabled* events, defined for all  $x \in \mathcal{X}$   
with  $\Gamma(x) \subseteq \mathcal{E}$

$p(x'; x, e')$  is a *state transition probability*, defined for all  $x, x' \in \mathcal{X}, e' \in \mathcal{E}$ ,  
and such that  $p(x'; x, e') = 0$  for all  $e' \notin \Gamma(x)$

$p_0(x)$  is the pmf  $P[X_0 = x]$ ,  $x \in \mathcal{X}$ , of the initial state  $X_0$

and  $G = \{G_i : i \in \mathcal{E}\}$  is a *stochastic clock structure*.

# Stochastic Timed Automata

- Next event  $E' = \arg \min_{i \in \Gamma(X)} \{Y_i\}$
- Next event time  $Y^* = \min_{i \in \Gamma(X)} \{Y_i\}$
- Next state  $X' = x'$  with probability  $p(x'; x, e')$
- Next timers 
$$Y'_i = \begin{cases} Y_i - Y^* & \text{if } i \neq E' \text{ and } i \in \Gamma(X) \\ V_{i, N_i + 1} & \text{if } i = E' \text{ or } i \notin \Gamma(X) \end{cases} \quad i \in \Gamma(X')$$
$$\{V_{i,k}\} \sim G_i$$

## Simulation Algorithm

### Main Loop

1. Process *current event* on top of the event list

*This may create additional events which will then have to be inserted in their proper place in the event list*

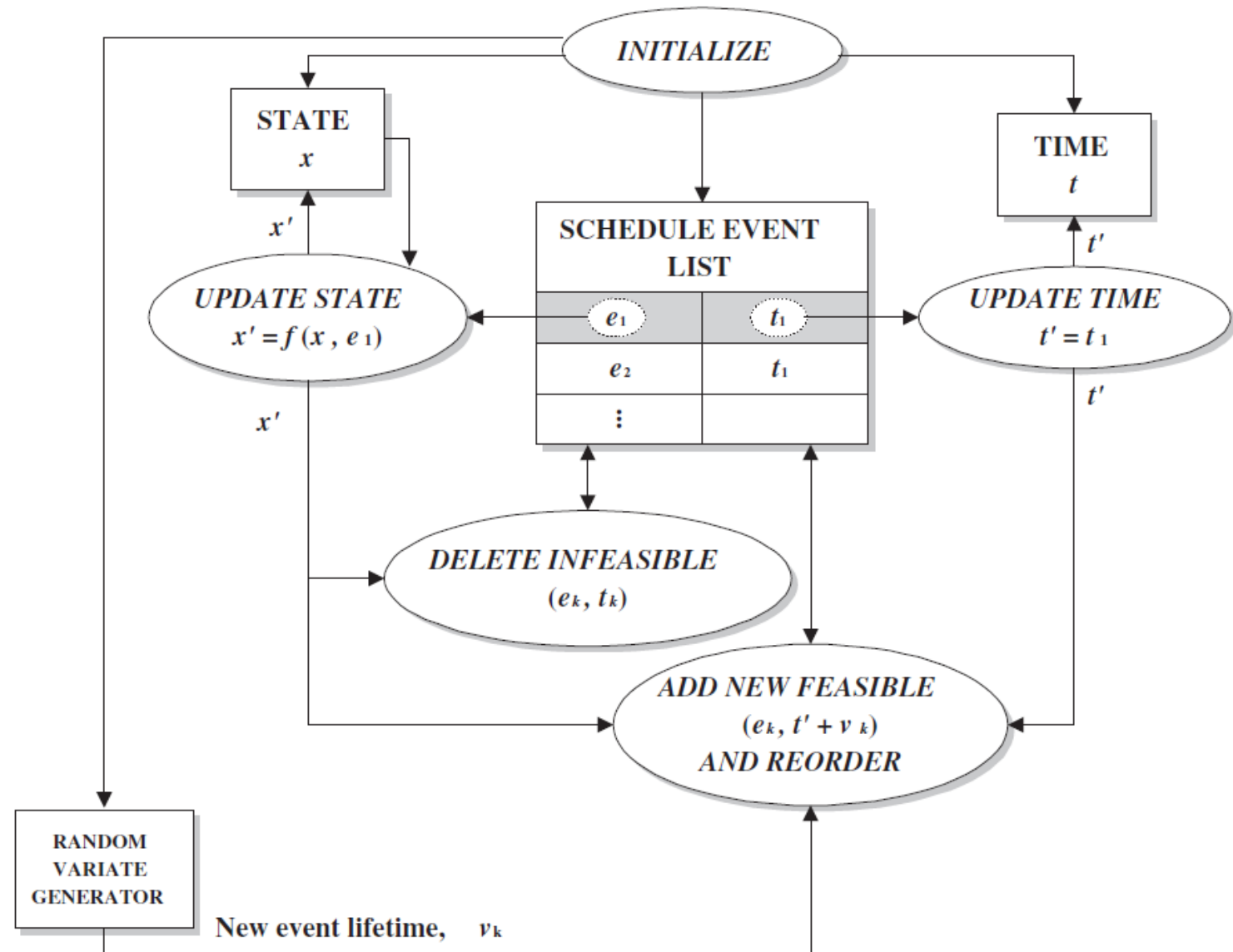
2. Move to the *next event* on the event list

3. Repeat from Step 1 until the *termination conditions* stipulated are reached

# Cassandras

- Step 1.* Remove the first entry  $(e_1, t_1)$  from the SCHEDULED EVENT LIST.
- Step 2.* Update the simulation TIME by advancing it to the new event time  $t_1$ .
- Step 3.* Update the STATE according to the state transition function,  $x' = f(x, e_1)$ .
- Step 4.* Delete from the SCHEDULED EVENT LIST any entries corresponding to infeasible events in the new state, that is, delete all  $(e_k, t_k) \in L$  such that  $e_k \notin \Gamma(x')$ .
- Step 5.* Add to the SCHEDULED EVENT LIST any feasible event which is not already scheduled (possibly including the triggering event removed in *Step 1*). The scheduled event time for some such  $i$  is given by  $(\text{TIME} + v_i)$ , where TIME was set in *Step 2* and  $v_i$  is a lifetime obtained from the RANDOM VARIATE GENERATOR.
- Step 6.* Reorder the updated SCHEDULED EVENT LIST based on a *smallest-scheduled-time-first* scheme.

# Cassandras

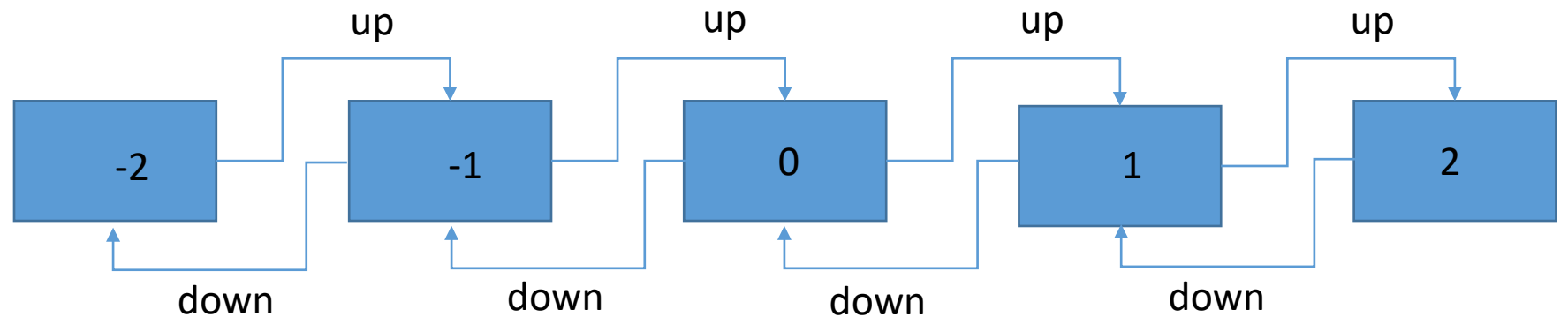


# Timing

- Continuous simulation time:  $t$
- Scheduling distribution:  $F(e,T)$  (function of event)
- Active event set:  $A(x)$  (function of state)
- Reset event set:  $R(x,e)$  (function of state and event) ( $R \subseteq A$ )
- Deactivated event list:  $D(x,e)$  (function of state and event)
- Loop:
  - De-queue event list top:  $(e^*,t^*)=dQ(\text{event list})$ ;
  - Update simulation time:  $t=t^*$
  - State transition  $x(n+1)=N(x(n),e^*)$
  - for each  $e$  in  $R(x(n+1),e^*)$ 
    - Pick  $T$  according to  $F(e)$
    - Event  $e$  scheduled at  $t+T$
    - Insert event in event list
  - For each  $e$  in  $D(x(n+1),e^*)$ 
    - Remove  $e$  from event list (or set scheduled time to infinity)

# Example cont'd

- $A(-1,..1)=\{\text{up},\text{down}\}$
- $A(-2)=\{\text{up}\}$
- $A(-1,..1)=\{\text{up},\text{down}\}$
- $A(2)=\{\text{down}\}$
- $D(-2)=\{\text{down}\}$
- $D(2)=\{\text{up}\} ??$



# Random selection – Inverse Function Method

- Assume  $F(e,T)$  is known and increasing in  $T$  whenever  $0 < F(e) < 1$
- Draw  $u$  randomly uniformly distributed in  $[0,1]$

Let  $T=F^{-1}(e,u)$

- $T$  is distributed according to  $F(e,.)$
- Proof

$$P(T \leq \tau) = P(F^{-1}(e, u) \leq \tau) = P(u \leq F(e, \tau)) = F(e, \tau)$$

# Example – exponential distribution

- Exponential distribution:  $F(t) = 1 - \exp(-\lambda t)$

$$u = 1 - \exp(-\lambda t)$$

$$1 - u = \exp(-\lambda t)$$

$$\log(1 - u) = -\lambda t$$

$$\frac{\log(1 - u)}{-\lambda} = t$$

# Other methods

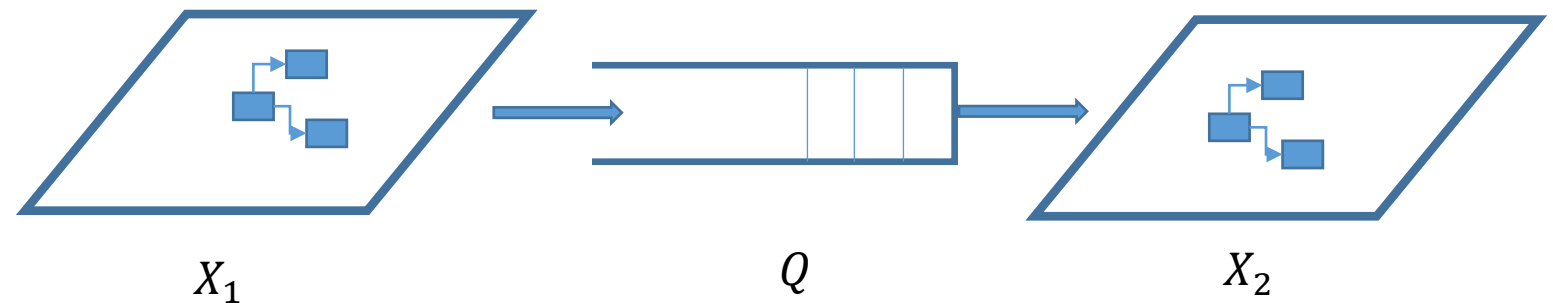
- For some distributions  $F(t)$  is not known
- Normal distribution
- Other methods that don't need distribution fct
  - Accept rejectance methods
  - Polar transformation for normal distribution

# Composed State Spaces

- In embedded systems a number of concurrent processes/tasks may be active, like tasks on processor sharing and nodes on a network
- Each process has its own state space  $X_i$
- Composed state space is Cartesian product of state spaces, i.e.
$$X = X_1 \times X_2 \times \dots \times X_N = \times_{i=1}^N X_i$$
- Cartesian product hold all N-tuples of state combinations, so all individual states are known from an element of the Cartesian product.
- Sometimes this is not enough

# Example – asynchronous communication

- Also  $Q$  content is part of the system state



$$X = X_1 \times Q \times X_2$$

- Asynchronous communication can increase the size of the state space significantly.

# A taxonomy of Markov Models in continuous time

- $x(n) \in \mathbb{R}^N$
- $\tau_n \in \mathbb{R}_+^M$  is the vector of event timers
- $T_n$  is the inter-arrival time between  $n$ th and  $(n+1)$  st event

- **Generalized Semi Markov Processes (GSMP):**

The sequence:  $\{(x(n), \tau_n)\}$  is a Discrete Time Markov Chain (DTMC)

- **Semi Markov Process**

We have generally:

$$P(x(n+1), T_n | x(n), T_{n-1}, \dots) = P(x(n+1), T_n | x(n))$$

Specifically: the sequence  $\{x(n)\}$  is a (DTMC)

- **Markov Process**

$$P(x(t_n) | x(t_{n-1}), x(t_{n-2}), \dots) = P(x(t_n) | x(t_{n-1}))$$

for any  $t_n > t_{n-1} > t_{n-2} > \dots$

# A taxonomy of Markov Models in continuous time

- **Generalized Semi Markov Process (GSMP):**
  - When more than 1 non-exponential timer are active simultaneously
  - Hard to analyze
  - Even output analysis after simulation may be hard
  - Markovian approximation
- **Semi Markov Process**
  - When only 1 or less non-exponential timers active simultaneously
  - Since state sequence is Markov all DTMC analysis tools work
  - Continuous time analysis may be added
  - Markovian approximation
- **Markov Process**
  - When all timers are exponential
  - All CTMC and DTMC tools may be applied (queueing theory)
  - Simulation only when state space is extremely large (rare event simulation).

# Stochastic Simulation of DTMC

- Select  $x(0) \in X$  (randomly or deterministically)
- for  $n=1:N-1$ 
  - select  $x(n + 1)$  (randomly) according to
$$P(x(n + 1) \in A_{n+1} | x(n) )$$

- Produces random sequence
$$\{x(n), n = 0, \dots, N - 1\}$$

- This sequence is amenable to statistics, e.g.

$$\hat{P}_N(A) = \frac{1}{N} \sum_{k=0}^{N-1} I_{x(n) \in A} = \frac{1}{N} \#\{x(n) \in A\}$$

- Registers the fraction of visits to subset  $A \subseteq X$
- Example: how often is the backlog  $x(n) \geq Q$

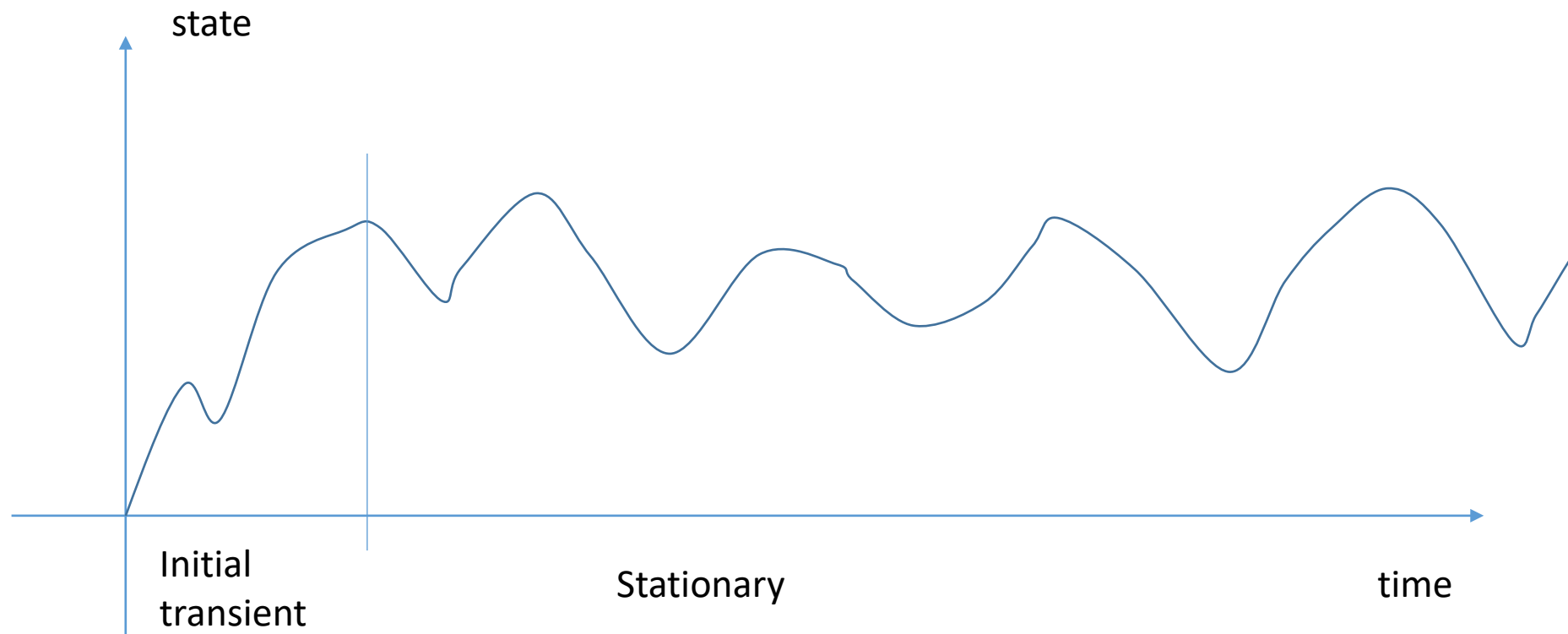
# Stochastic Simulation of CTMC/GSMP

- Select  $x(0) \in X$  (randomly or deterministically)
- for  $n=1:N-1$ 
  - select  $x(n+1)$  and timers  $\tau_n$  (randomly) according to
$$P(x(n+1), \tau_{n+1} | x(n), \tau_n)$$
- Produces random piecewise constant realization
$$\{x(t), t \in [t_A, t_B]\}$$
- This realization is (might be) amenable to statistics, e.g.

$$\hat{P}_N(A) = \int_{t_A}^{t_B} I_{x(\tau) \in A} d\tau$$

- Registers the fraction of time where the state is in  $A \subseteq X$
- Example: how much of the time is the backlog  $x(t) \geq Q$

# Initial transient and stationary behaviour



# Restarted and regenerative simulation

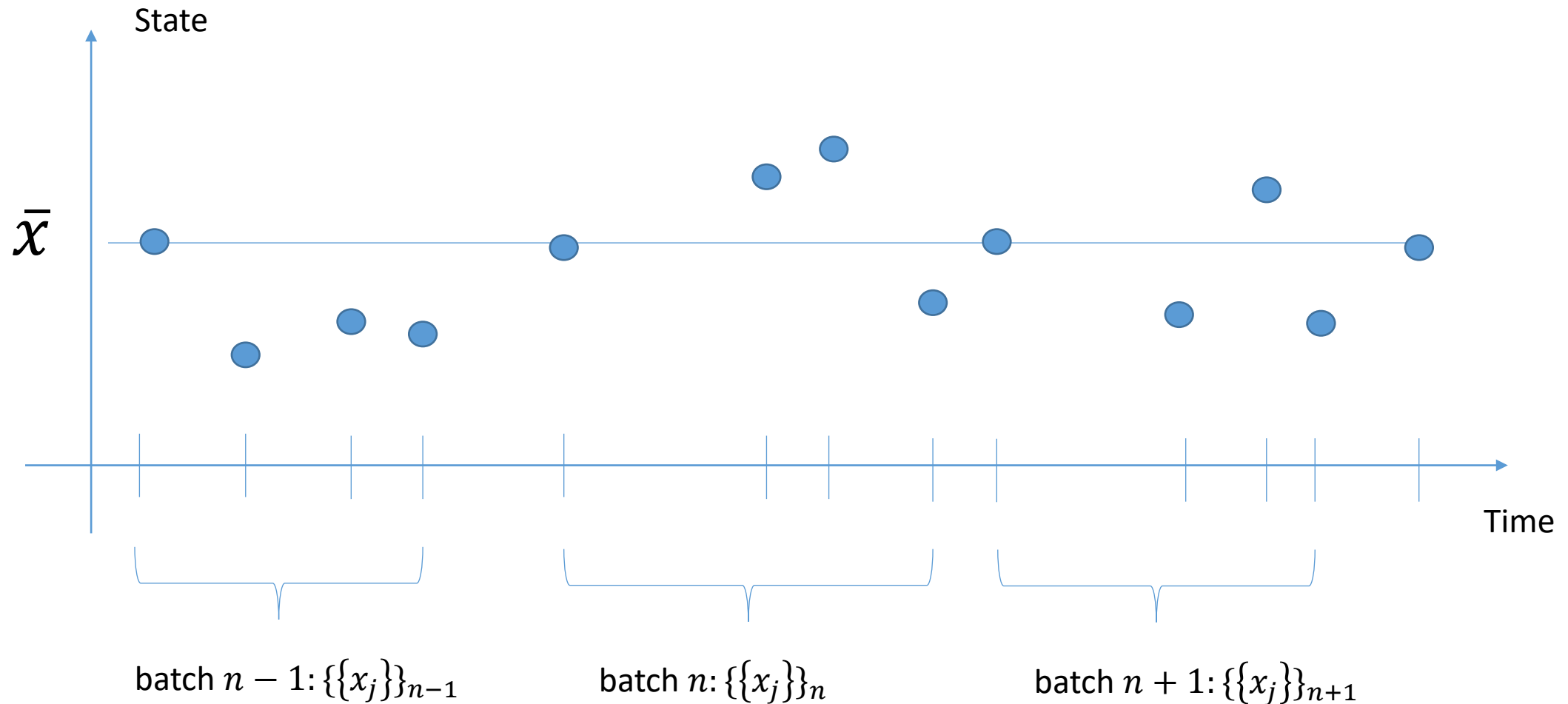
- Restarted:
  - Simulation is restarted a number of times  $M$  with a fixed sequence length  $N_{res}$  or until some state is reached
  - This is feasible for obtaining *transient* statistics
  - The  $M$  runs/restarts may be considered statistically independent in the subsequent statistical analysis
- Regenerative:
  - Simulation is started **once** with a fixed sequence length  $N_{reg} \gg N_{res}$
  - This is feasible for obtaining *long-term* statistics
  - Subsequent statistical analysis is more complex – depends on *regeneration*

# Output Analysis

# Regeneration for DTMC

- Let  $\{x(n)\}$  be a DTMC
- Let  $J = \{i \mid x(i) = \bar{x}\}$
- Let time intervals  $I_k$  be defined by  $I_k = \{J_k, \dots, J_{k+1} - 1\}$
- Define statistics  $s_k = S(x(J_k), \dots, x(J_{k+1} - 1))$
- Then, the sequence  $\{s_k\}$  is an i.i.d. sequence
- i.i.d. property makes statistics easier.
- Easy mean and variance computations
- $\bar{x}$  should have frequent revisits
- $\bar{x}$  generally not easy to find in GSMP since  $\{(x(n), \tau_n)\}$  is the DTMC

# Independent batches



# Examples

- Average state value:

$$f(x) = x$$

- Frequency of occurrence:

$$f(x) = I_{x \in A}$$

# Batch statistics

- Independent batches:  $\{\{x_j\}\}_n$

- Consider statistic:

$$S_N = \frac{1}{N} \sum_{j=0}^{N-1} f(x_j)$$

- Break up into batches:

$$S_N = \frac{1}{N} \sum_n \sum_{x \in \{x_j\}_n} f(x) = \frac{1}{N} \sum_n B_n$$

- Batch statistics  $B_n$  are statistically independent and identically distributed

$$\text{var}(S_N) = \frac{1}{N} \text{var}(B_n)$$

- Variance of  $(B_n)$  needs to be estimated:

$$\hat{V}_B = \frac{1}{N} \sum_n (B_n - S_N)^2$$

# Examples

- Time average of state value (T is interarrival time):

$$f(T, x) = T x$$

- Time fraction of occurrence:

$$f(T, x) = T I_{x \in A}$$

# Batch statistics

- Independent batches:  $\{\{x_j\}\}_n$
- Consider statistic:

$$S_N = \frac{1}{N} \sum_{j=0}^{N-1} f(T_j, x_j)$$

- Break up into batches:

$$S_N = \frac{1}{N} \sum_n \sum_{(T,x) \in \{\{(T_j, x_j)\}\}_n} f(T, x) = \frac{1}{N} \sum_n B_n$$

- Batch statistics  $B_n$  are statistically independent and identically distributed

$$\text{var}(S_N) = \frac{1}{N} \text{var}(B_n)$$

- Variance of  $B_n$  needs to be estimated:

$$\hat{V}_B = \frac{1}{N} \sum_n (B_n - S_N)^2$$

# Restarted simulation – example (Poisson counting in TrueTime)

```
function [exectime, dataL] = generator_code(seg,dataL)
    Q=Q+1; %increment counter
    ttAnalogOut(1, Q); %show counter on analog output
    %select next arrival instant randomly with exp. Interarrival time
    u=rand(1); %uniformly distributed random variable
    tnxt=-log(1-u)/lambda; %use inverse fct method
    ttCreateJob('generator_task', ttCurrentTime+tnxt); %schedule next job
    exectime = -1;
end
```

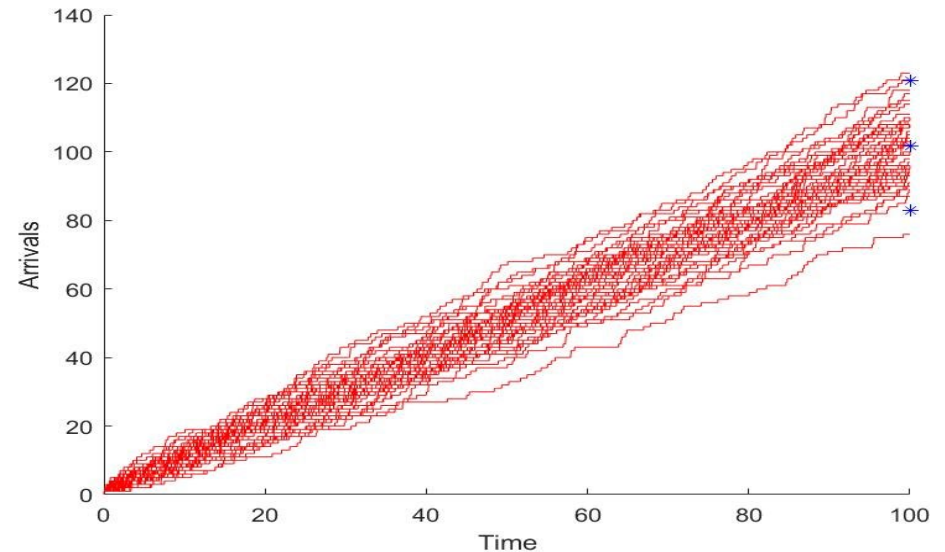
# Restart script in Matlab

```
M=50; %number of restarts
N=100; %Simulation time

SimData={}; %list for data
for i=1:M
    sim('MM1',N); %run SimuLink model for specific time N
    SimData{i}=[simout.Data simout.Time]; %record data in list
end

%statistical/output analysis
%Find mean of end value after time N
E=0;
clf
hold on
for i=1:M
    E=E+SimData{i}(end,1);
end
m=E/M %estimate mean value
```

```
V=0;
for i=1:M
    V=V+(SimData{i}(end,1)-m)^2;
end
var=V/M %estimate variance of batch values
var=V/M^2 %estimate variance of estimate E
stdd=sqrt(var) %estimate standard deviation
```



Restarted realizations of Poisson counting process (in red).  
Estimated mean with 2\*std added and subtracted (blue \*).

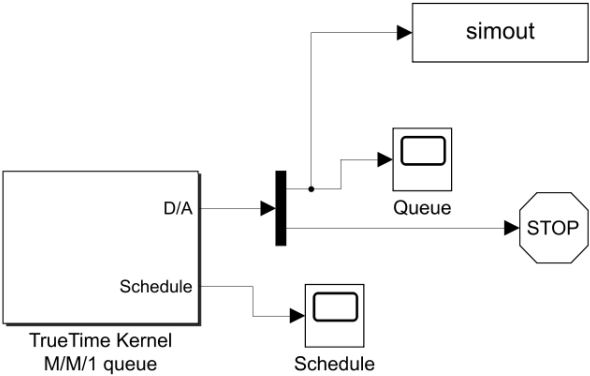
# Regenerative Simulation – example

## MM1 queue

```
function [exetime, dataL] = generator_code(seg,dataL)
    Q=Q+1;
    ttAnalogOut(1, Q);
    u=rand(1);
    tnxt=-log(1-u)/lambda; ttCreateJob('generator_task', ..
                                      ttCurrentTime+tnxt);
    exetime = -1;
end
```

```
function [exetime, dataL] =
service_code(seg,dataL)

if(Q>0)
    Q=Q-1;
end
ttAnalogOut(1, Q);
if(Q>0)
    u=rand(1);
    tnxt=-log(1-u)/mu;
    ttCreateJob('service_task', ..
              ttCurrentTime+tnxt);
end
if(Q==0) %selected regeneration state
    ttAnalogOut(2, 1); %send stop signal to SimuLink
end
exetime = -1;
```



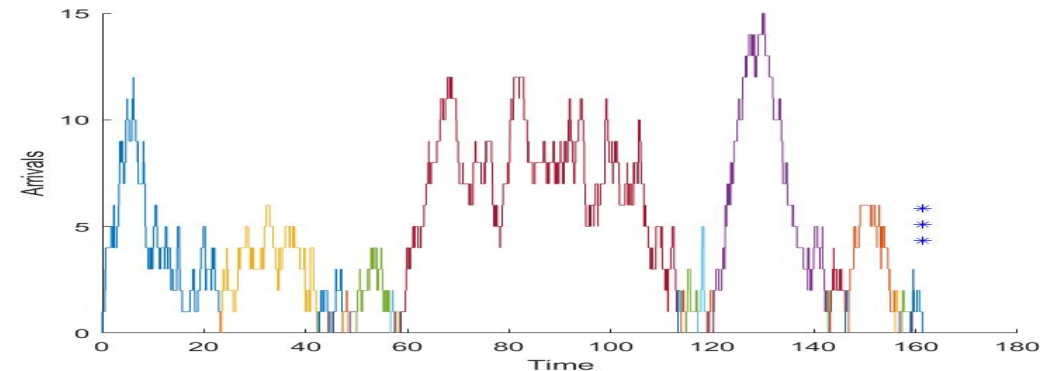
The diagram illustrates the simulation logic for an M/M/1 queue. It features a 'TrueTime Kernel M/M/1 queue' block containing a 'D/A' (Digital-to-Analog) converter and a 'Schedule' block. The 'D/A' block outputs to a 'Queue' block, which then feeds into a 'simout' block. The 'Schedule' block outputs to a 'Schedule' block, which then feeds into a 'STOP' block. The 'Queue' block also feeds into the 'Schedule' block. The 'simout' block outputs to the 'Queue' block. The 'STOP' block is a terminal state.

# Regeneration script in Matlab

```
SimData={}; %list for data
for i=1:M
    sim('MM1',N); %run SimuLink model for specific time N
    SimData{i}=[simout.Data simout.Time]; %record data in list
end

%Find mean of time integral of backlog Q(t)
%corresponds to MM1 result  $Q=r/(1-r)$ 
E=0;
plottime=0;
for i=1:M
    %compute time increments {T}
    T=SimData{i}(2:end,2)-SimData{i}(1:end-1,2);
    E=E+(T'*SimData{i}(1:end-1,1));
    plottime=plottime+SimData{i}(end,2);
end
mT=E/plottime %estimate time average
m=E/M; %estimate mean of integrals
```

```
V=0;
for i=1:M
    T=SimData{i}(2:end,2)-SimData{i}(1:end-1,2);
    V=V+(T'*SimData{i}(1:end-1,1)-m)^2;
end
var=V/M; %estimate variance batch samples
varm=var/M^2 %estimate variance of m
varT=varm*(M^2/plottime^2) %estimate variance of
time average
stddev=sqrt(varT) %estimate standard deviation
```



Realization of MM1 simulation divided into batches (different colors) by revisits to state ( $Q=0$ ).  
Estimated mean with  $2 \times \text{std}$  added and subtracted (blue \*).

# Various Markov models

- GSMP

$$P(x(n+1) \in A_{n+1}, \tau_{n+1} \in T_{n+1} | x(n), \tau_n) = F(x(n), \tau_n)$$

$\{(x(n), \tau_n)\}$  is a DTMC

Regeneration at specific state  $(x, \tau)$

Very unlikely, since state-space is very complex

- All exponential timers:

$$P(x(n+1) \in A_{n+1} | x(n), x(n-1), \dots) = P(x(n+1) \in A_{n+1} | x(n)) = F(x(n))$$

$\{x(n)\}$  is a DTMC

Regeneration at specific state  $x$  – since timers are reset

More likely

# Reset for exponential timer

(proof at the end of the slide)

Theorem:

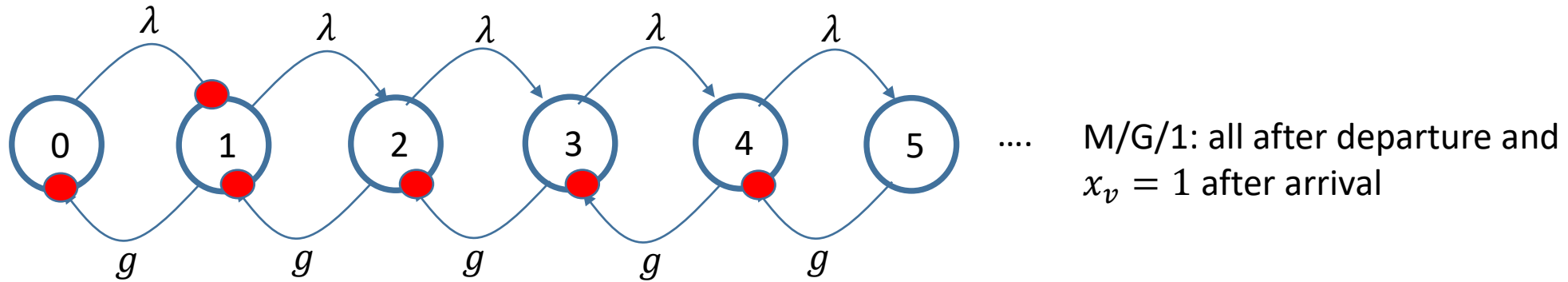
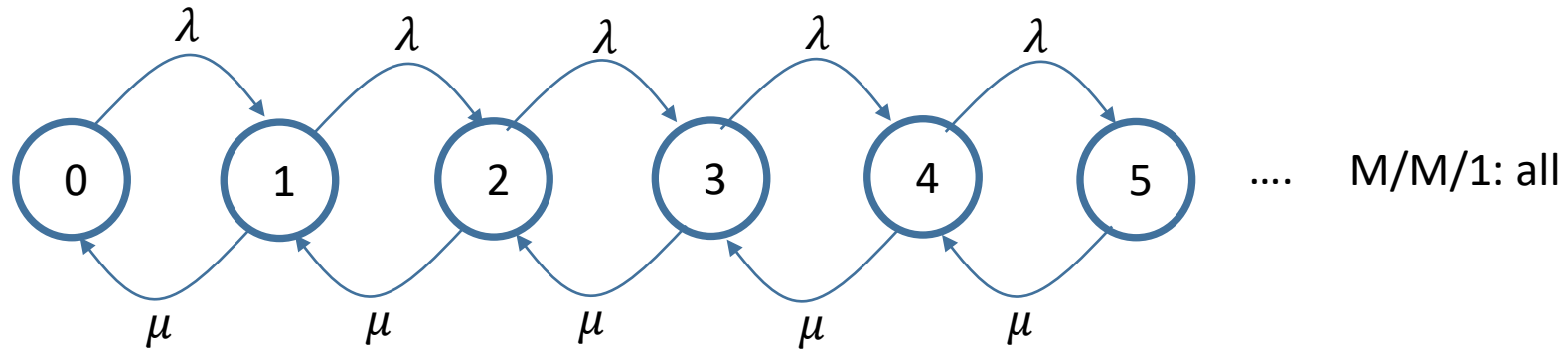
- Let  $t_n$  the transition time for the transition to  $x(n)$
- Let  $e \in (A/R)UD$  be exponential
- Let  $t_e^R$  be the latest reset time for  $e$ , i.e.  $t_e^R < t_n$
- Let  $t_e^T$  be the active timer value for  $e$ , ie.  $t_e^T = T + t_e^R$
- Where  $T$  is independent, exponential with parameter  $\lambda_e$
- Define  $\hat{t} = (t_e^T \leq t_n)t_e^T + (t_e^T > t_n)(t_n + \hat{T})$
- Where  $\hat{T}$  is exponential (independent) with parameter  $\lambda_e$
- Then

$\hat{t}_n - t_e^R$  is independent exponential

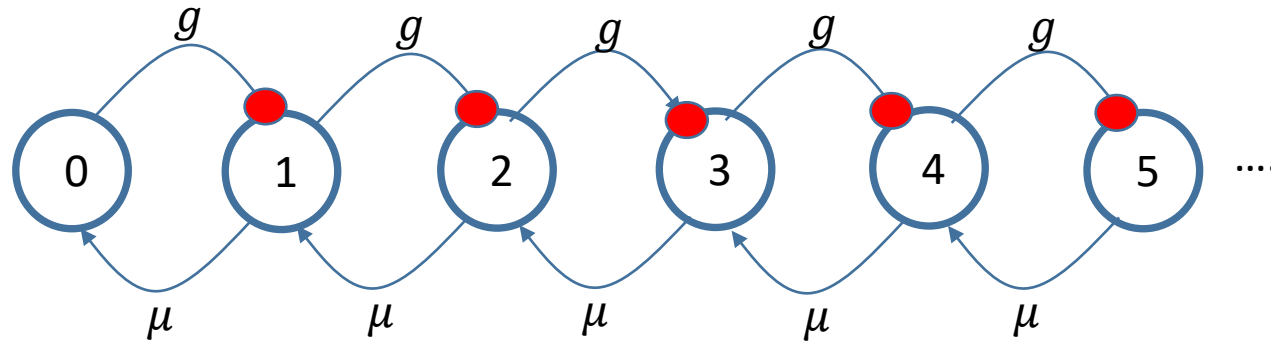
# Selection of re-generation state $\bar{x}$ (various queueing models)

- $\bar{x}$  should be visited often to produce many independent batches
- $\bar{x}$  includes also buffer states (content)
- M/M/1: every state is possible re-generation state no matter the event (arrival, departure)
- M/G/1: every state is a possible re-generation state, when entered through departure (general service timer restarted).
- G/M/1: every state is a possible re-generation state, when entered through arrival (general arrival timer restarted).
- G/G/1: state  $\bar{x} = 1$  is possible re-generation state when entered through arrival (since both general timers are restarted).

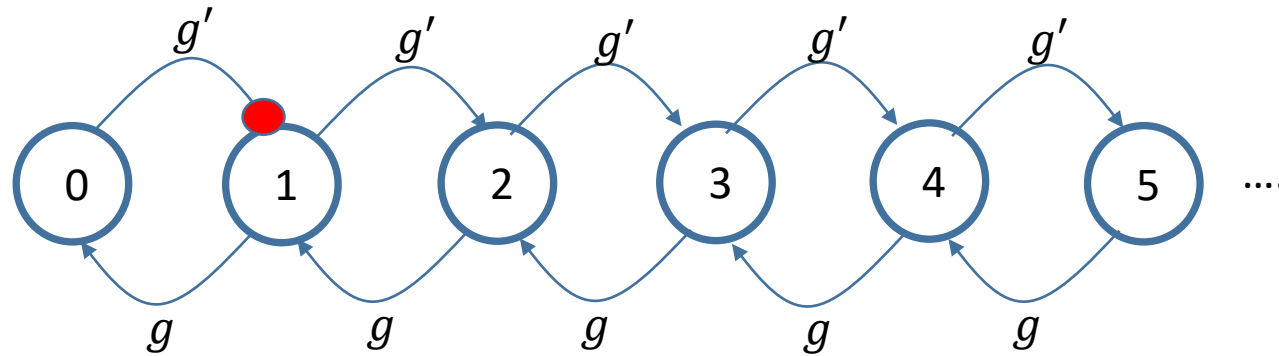
## Selection of re-generation state $\bar{x}$



# Selection of re-generation state $x_v$



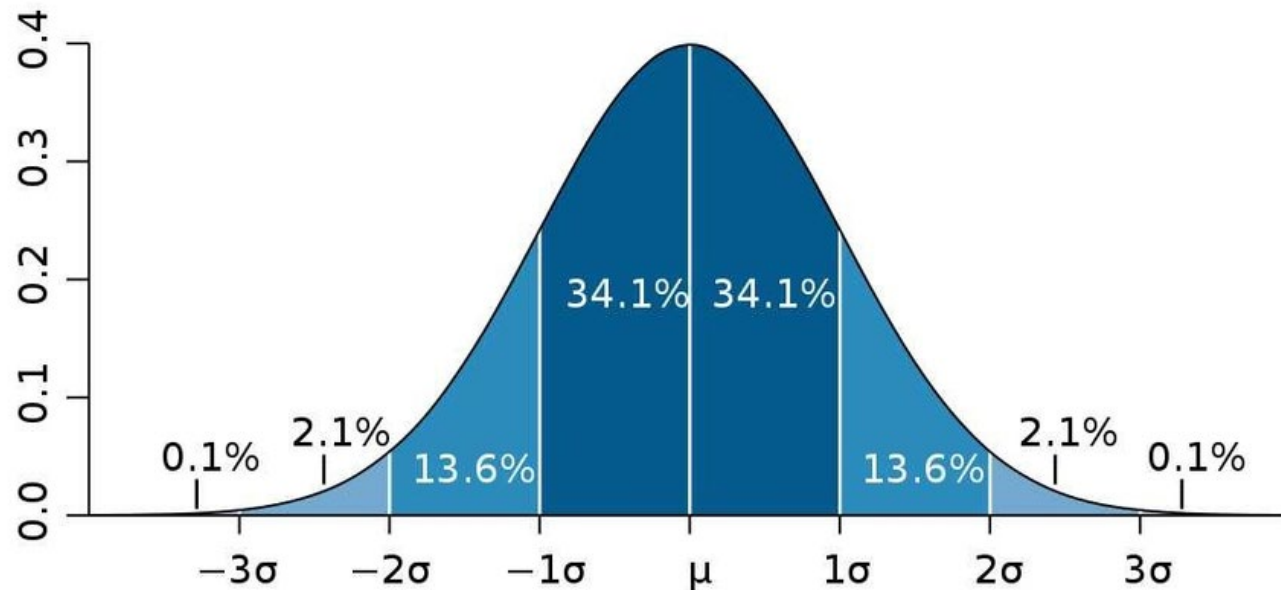
G/M/1: all after arrival



G/G/1:  $x_v = 1$  after arrival and

# (Confidence) Interval estimation

- Through the Central Limit Theorem all sums may be assumed normal
- With a credible variance(std) estimate confidence are easily computed
- $2 \times \sigma$  gives two sides 95% confidence interval



# Reset for exponential timer

(all proofs may be omitted during reading)

Theorem:

- Let  $t_n$  the transition time for the transition to  $x(n)$
- Let  $e \in (A/R)UD$  be exponential
- Let  $t_e^R$  be the latest reset time for  $e$ , i.e.  $t_e^R < t_n$
- Let  $t_e^T$  be the active timer value for  $e$ , ie.  $t_e^T = T + t_e^R$
- Where  $T$  is independent, exponential with parameter  $\lambda_e$
- Define  $\hat{t} = (t_e^T \leq t_n)t_e^T + (t_e^T > t_n)(t_n + \hat{T})$
- Where  $\hat{T}$  is exponential (independent) with parameter  $\lambda_e$
- Then

$\hat{t}_n - t_e^R$  is independent exponential

# Interpretation of reset Theorem

- We compare

$$t_e^T - t_e^R = T + t_e^R - t_e^R = T \text{ and } \hat{t}_n - t_e^R$$

- Since they are both independent and exponential we may replace

$$t_e^T \text{ with } \hat{t}_n$$

- With

$$\hat{t} = (t_e^T \leq t_n)t_e^T + (t_e^T > t_n)(t_n + \hat{T})$$

- If  $t_e^T$  is before  $t_n$  we just use  $t_e^T$
- If not we restart the timer and use  $(t_n + \hat{T})$
- Thus, every exponential timer may be considered restarted at every transition instant.

# Proof of reset Theorem

- Define  $\hat{t} = (t_e^T \leq t_n)t_e^T + (t_e^T > t_n)(t_n + \hat{T})$
- Where  $\hat{T}$  is exponential with parameter  $\lambda_e$
- Fix  $t_n$  and (let all probabilities below be conditional on  $t_n$ )
- Then

$$\begin{aligned} P(\hat{t} \leq z) &= P\left((t_e^T \leq t_n)t_e^T + (t_e^T > t_n)(t_n + \hat{T}) \leq z\right) = \\ &P\left((t_e^R + T \leq t_n)t_e^T + (t_e^R + T > t_n)(t_n + \hat{T}) \leq z\right) = \end{aligned}$$

$$\begin{aligned} &P(T \leq t_n - t_e^R)P(t_e^R + T \leq z \mid t_e^R + T \leq t_n) \\ &+ P(t_e^R + T > t_n)P(t_n + \hat{T} \leq z \mid t_e^R + T > t_n) = \end{aligned}$$

$$\begin{aligned} &P(T \leq t_n - t_e^R)P(T \leq z - t_e^R \mid T \leq t_n - t_e^R) + \\ &P(T > t_n - t_e^R)P(\hat{T} \leq z - t_n \mid T > t_n - t_e^R) \end{aligned}$$

# Proof of reset Theorem

$$\begin{aligned} & P(\hat{t} \leq z) \\ &= \\ & P(T \leq t_n - t_e^R)P(t_e^R + T \leq z \mid t_e^R + T \leq t_n) + P(t_e^R + T > t_n)P(t_n + \hat{T} \leq z \mid t_e^R + T > t_n) \\ &= \\ & P(T \leq t_n - t_e^R)P(T \leq z - t_e^R \mid T \leq t_n - t_e^R) + P(T > t_n - t_e^R)P(\hat{T} \leq z - t_n \mid T > t_n - t_e^R) \end{aligned}$$

$$\begin{aligned} & P(\hat{t} - t_e^R \leq z) = P(\hat{t} \leq z + t_e^R) \\ &= \\ & P(T \leq t_n - t_e^R)P(T \leq z + t_e^R - t_e^R \mid T \leq t_n - t_e^R) + P(T > t_n - t_e^R)P(\hat{T} \leq z + t_e^R - t_n \mid T > t_n - t_e^R) \\ &= \\ & P(T \leq t_n - t_e^R)P(T \leq z \mid T \leq t_n - t_e^R) + P(T > t_n - t_e^R)P(\hat{T} \leq z + t_e^R - t_n \mid T > t_n - t_e^R) \\ &= \\ & (1 - \exp(t_e^R - t_n))(t_n - t_e^R \leq z) + (1 - \exp(-z))(t_n - t_e^R > z) \\ & \quad + \exp((t_e^R - t_n))(1 - \exp(-(z + t_e^R - t_n)))(0 \leq z + t_e^R - t_n) \end{aligned}$$

# Proof of reset Theorem

$$\begin{aligned} & P(\hat{t} - t_e^R \leq z) \\ &= \\ & (1 - \exp(t_e^R - t_n)) (t_n - t_e^R \leq z) + (1 - \exp(-z)) (t_n - t_e^R > z) \\ & + \exp(-(t_e^R - t_n)) (1 - \exp(-(z + t_e^R - t_n))) (0 \leq z + t_e^R - t_n) \\ &= \\ & (1 - \exp(t_e^R - t_n)) (t_n - t_e^R \leq z) + (1 - \exp(-z)) (t_n - t_e^R > z) \\ & + \exp((t_e^R - t_n)) (1 - (\exp(-z) \exp(t_n - t_e^R))) (0 \leq z + t_e^R - t_n) \\ &= \\ & (1 - \exp(t_e^R - t_n)) (t_n - t_e^R \leq z) + (1 - \exp(-z)) (t_n - t_e^R > z) \\ & + (\exp((t_e^R - t_n)) - \exp(-z)) (t_n - t_e^R \leq z) \end{aligned}$$

# Proof of reset Theorem

$$\begin{aligned} & P(\hat{t} - t_e^R \leq z) \\ &= \\ & (1 - \exp(t_e^R - t_n)) (t_n - t_e^R \leq z) + (1 - \exp(-z)) (t_n - t_e^R > z) \\ & \quad + (\exp((t_e^R - t_n)) - (\exp(-z))) (t_n - t_e^R \leq z) \\ &= \\ & (t_n - t_e^R \leq z) - \exp(t_e^R - t_n) (t_n - t_e^R \leq z) \\ & \quad + (t_n - t_e^R > z) - \exp(-z) (t_n - t_e^R > z) \\ & \quad + \exp((t_e^R - t_n)) (t_n - t_e^R \leq z) - \exp(-z) (t_n - t_e^R \leq z) \\ &= 1 - \exp(-z) \end{aligned}$$

# Proof of Markov property for exponential timers

- Generally:

$$P(x(n+1) \in A_{n+1}, \tau_{n+1} \mid x(n), x(n-1), \dots, \tau_n) = F(x(n), \tau_n)$$

and through marginalization of  $\tau_{n+1}$  :

$$\begin{aligned} &P(x(n+1) \in A_{n+1} \mid x(n), x(n-1), \dots, \tau_n) \\ &= \\ &G(x(n), \tau_n) = P(x(n+1) \in A_{n+1} \mid x(n), \tau_n) \end{aligned}$$

# Proof of Markov property for exponential timers

$$P(x(n+1) \in A_n | x(n), \dots)$$

$$= \int_{\tau_n} P(x(n+1) \in A_n | x(n), \dots, \tau_n) P(\tau_n | x(n), \dots) d\tau_n$$

$$= \int_{\tau_n} F(x(n), \tau_n) P(\tau_n | x(n), \dots) d\tau_n$$

# Proof of Markov property for exponential timers

- Now if

$$P(\tau_n | x(n), \dots) = P(\tau_n | x(n)) \quad (*)$$

- Then

$$\begin{aligned} P(x(n+1) \in A_n | x(n), \dots) \\ &= \int_{\tau_n} F(x(n), \tau_n) P(\tau_n | x(n), \dots) d\tau_n \\ &= \int_{\tau_n} F(x(n), \tau_n) P(\tau_n | x(n)) d\tau_n = F(x(n)) \end{aligned}$$

- If all timers are reset or deactivated (\*) will be the case, since timer values will be determined by  $x(n)$  alone